Programming Fundamentals

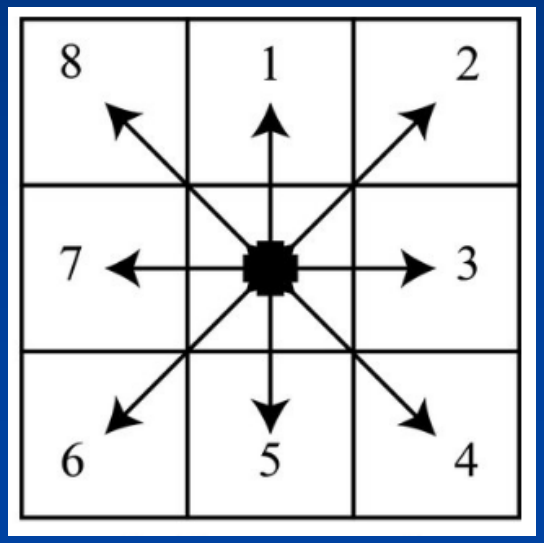
- Lecturer: Dr. Arnold Meijster
- e-mail: a.meijster@rug.nl
- phone: 050 363 9246
- room: 5161.343 (Bernoulliborg)

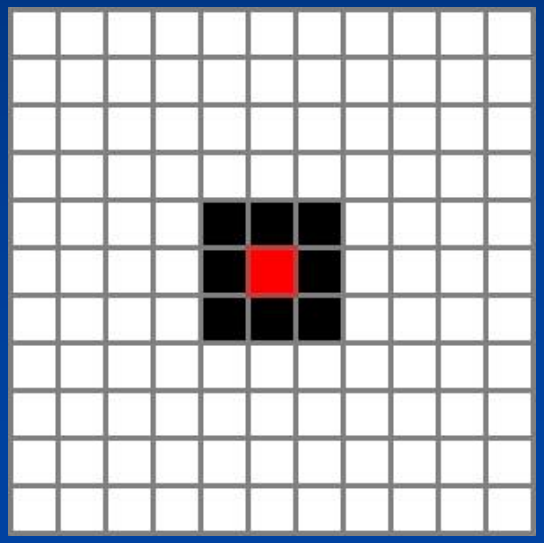


John Conway's Game of Life

Each cell has 8 neighbors

- 4 adjacent orthogonally
- 4 adjacent diagonally.
- This is called 8-connectivity / chesboard neighborhood.





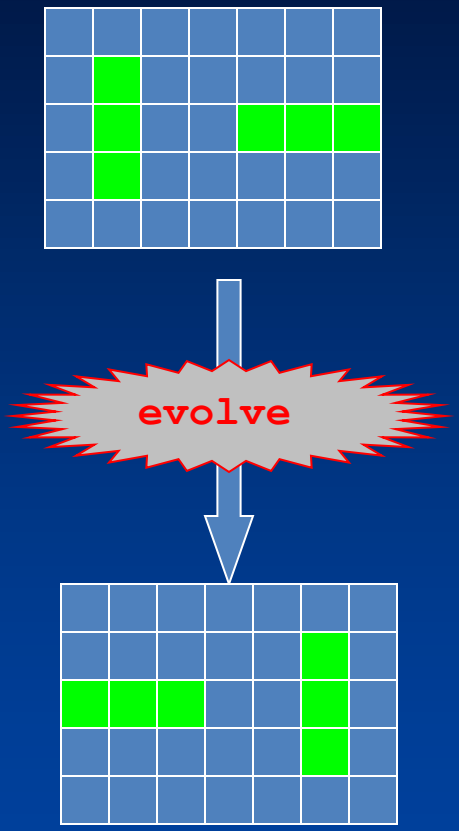
Game of Life: Simple rules, executed at each time step:

A living cell with 2 or 3 alive neighbors survives to the next round.

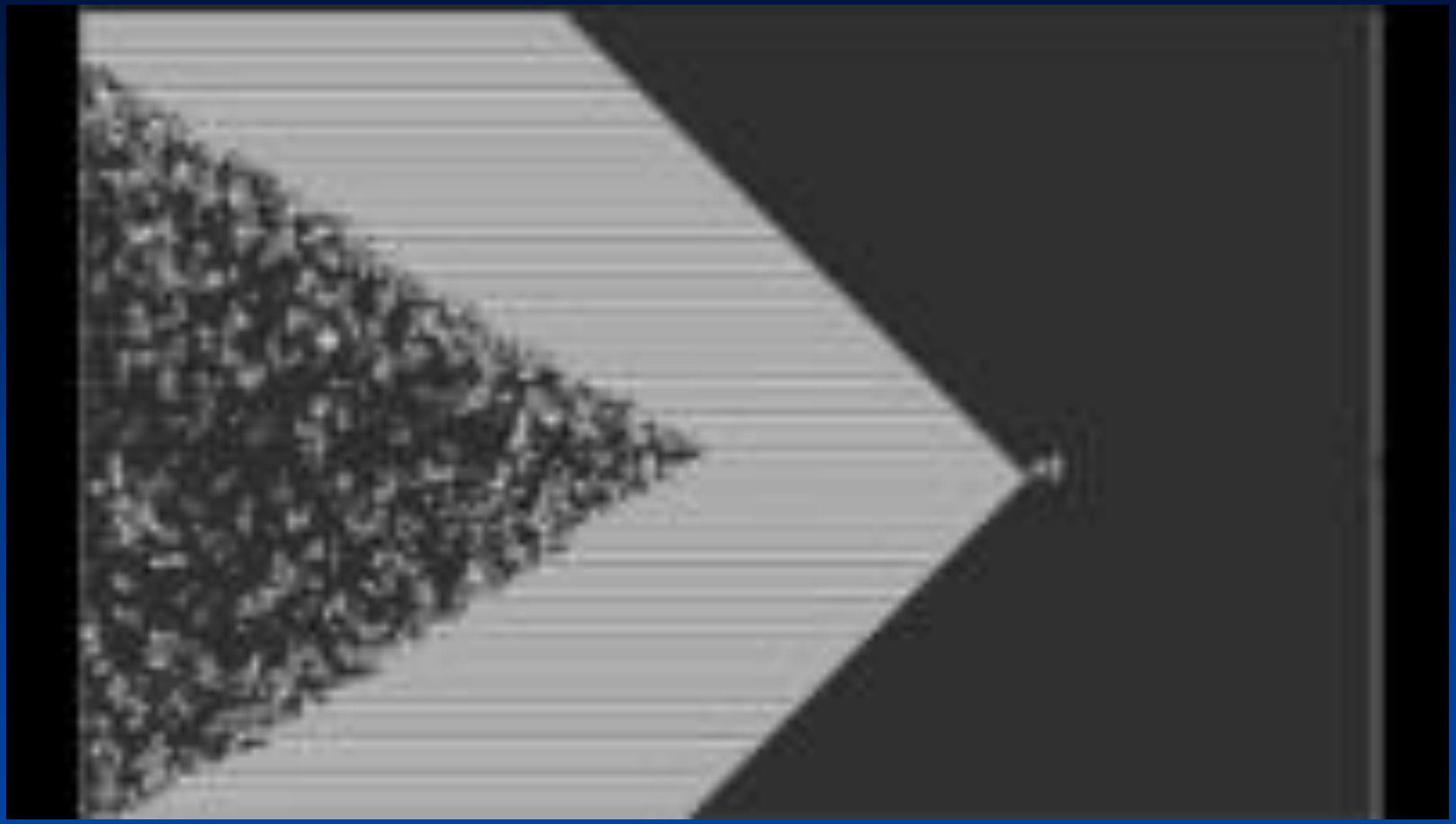
A living cell with 4 or more neighbors dies of overpopulation.

A living cell with 1 or 0 neighbors dies of isolation.

An empty cell with exactly 3 neighbors becomes a living cell in the next round.



Game of Life: video



John Conway's Game of Life

```
for (int row=0; row < n; row++) {
  for (int column=0; column < n; column++) {
    int neighbors = 0;
    for (int r=-1; r <= 1; r++) {
      for (int c=-1; c <= 1; c++) {
        if ((0 <= row + r) && (row + r < n) &&
            (0 <= column + c) && (column + c < n)) {
          neighbors += grid[row+r][column+c];
        }
      }
    }
    / * correction */
    neighbors -= grid[row][column];  /* subtract 1 if alive */
    /* compute next[row][column*/
    if (grid[row][column] == 0) {
      next[row][column] = (neighbors == 3);
    } else {
      next[row][column] = ((neighbors == 2) || (neighbors == 3));
    }
  }
}
```

sizeof

```
sizeof(typeName)
```

or

```
sizeof expression     [not advised, use (..)]
```

returns the size in bytes of the data type (or the expression).

```
sizeof(int)       /* 4 on PCs  */
sizeof(i*3)       /* 4 on PCs  */
sizeof i*3        /* 12 on PCs */
```

Sizeof: 32 bit PC sizes

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
  printf("sizeof(char)   = %ld\n", sizeof(char));
  printf("sizeof(short)  = %ld\n", sizeof(short));
  printf("sizeof(int)    = %ld\n", sizeof(int));
  printf("sizeof(long)   = %ld\n", sizeof(long));
  printf("sizeof(float)  = %ld\n", sizeof(float));
  printf("sizeof(double)= %ld\n", sizeof(double));
  return 0;
}
```

Output:

```
sizeof(char)   = 1
sizeof(short)  = 2
sizeof(int)    = 4
sizeof(long)   = 4
sizeof(float)  = 4
sizeof(double) = 8
```

Sizeof: 64 bit PC sizes

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
  printf("sizeof(char)   = %ld\n", sizeof(char));
  printf("sizeof(short)  = %ld\n", sizeof(short));
  printf("sizeof(int)    = %ld\n", sizeof(int));
  printf("sizeof(long)   = %ld\n", sizeof(long));
  printf("sizeof(float)  = %ld\n", sizeof(float));
  printf("sizeof(double)= %ld\n", sizeof(double));
  return 0;
}
```

Output:

```
sizeof(char)   = 1
sizeof(short)  = 2
sizeof(int)    = 4
sizeof(long)   = 8
sizeof(float)  = 4
sizeof(double) = 8
```



Grades

Assignment: write a program fragment that reads the grades of a number of students from the input and then prints a list with the grade of every student and the deviation relative to the mean grade.

Grades: plan

We number the students, starting with 0.

We ask the user to first type the number of students.

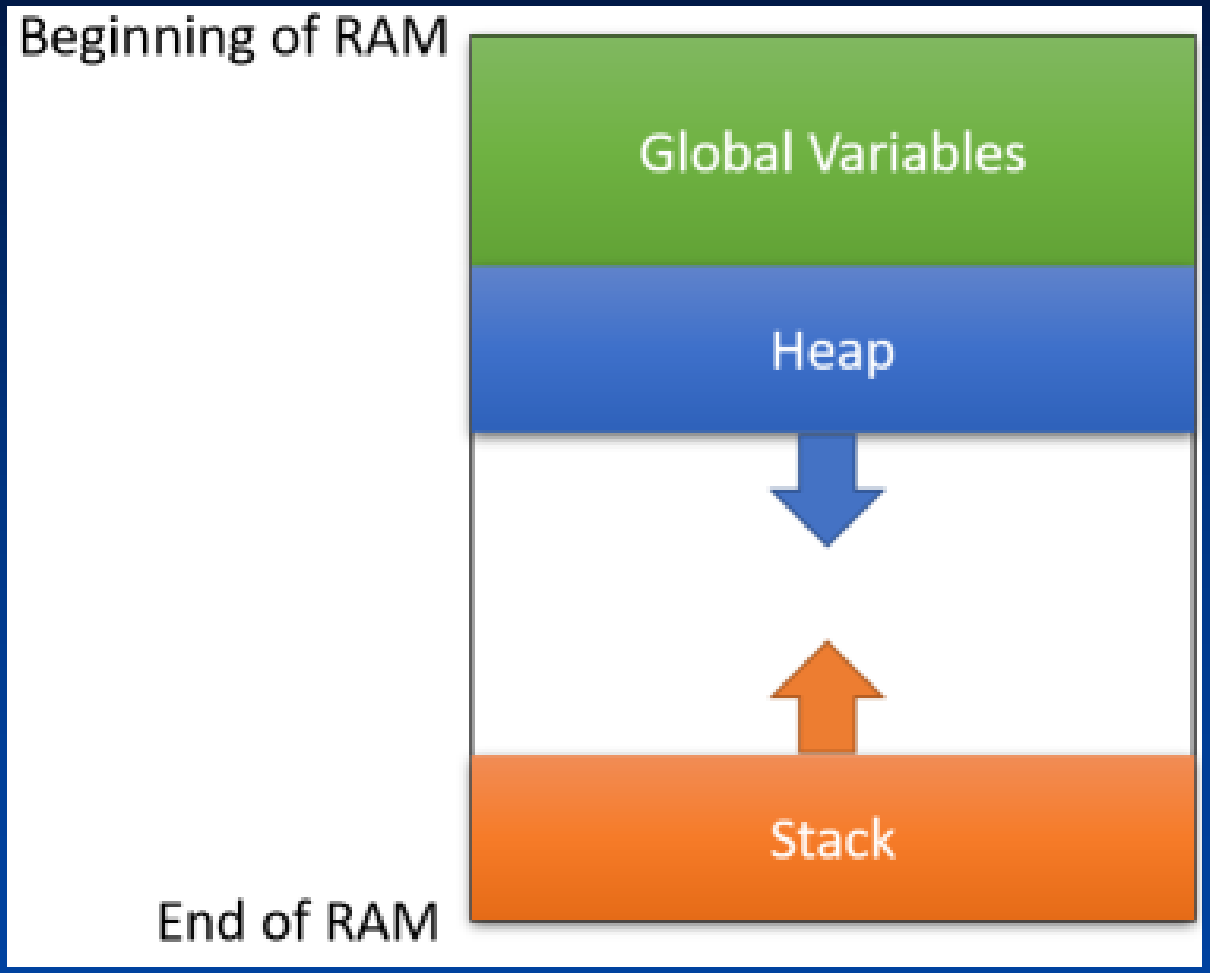
We create an array that has the right size. ???

Read the grades from the input.

Compute the mean.

Print the list with grades and deviations.

Intermezzo: Stack and Heap Memory



Intermezzo: Stack and Heap Memory

Stack (memory): *implicit* memory usage taken care of by the compiler.

Used for function calls, parameters and return values of functions.

Local variables are also stored on the stack.

The compiler is responsible for memory management.

Deallocation (‘clean up’) is automatic!

Intermezzo: Stack and Heap Memory

Heap memory: *explicit* memory usage by the programmer.

The programmer is responsible for heap memory management.

Sloppy memory management leads to ***memory leakage.***

Deallocation (‘clean up’) is not automatic!

`void` and `NULL`-pointer

`void *p;` declares a *generic,* or *typeless* pointer `p`.

- Commonly called a void-pointer.

Before usage, a void-pointer is cast to a typed pointer: `(T *)p`

- **For example:** `int x = *((int *)p);`

A void-pointer can be assigned to a typed pointer without casting, and vice versa.

- **For example:** `int *iptr = p;`

The NULL-pointer is a pointer to “any” data type.

- It is called a *generic* pointer.
- Its type is `void *`.
- `NULL` is actually defined as `(void *)0`.

Generic Pointers

```
int i = 2;
int *ip = &i;
void *p;

p = ip;
printf("%d", *p);  /* wrong */

printf("%d", *((int*)p));  /* ok */
```

Heap Memory Management

There are 2 functions that can be used for ***dynamic memory allocation***:

```
void *malloc(int requestedSize);
void *calloc(int requestedCount, int requestedSize);
```

```
T *p;

p = malloc(sizeof(T));      /* or */
p = calloc(1, sizeof(T));
```

The block of memory returned by **`malloc`** is unitialized! The block of memory returned by **`calloc`** is cleared (filled) with zero bytes.

Always check whether allocation was successful or failed. If the allocation failed, then the returned value is **`NULL`**!

Memory Allocation: handy

```
void *safeMalloc(int n) {
 void *p = malloc(n);
 if (p == NULL) {
   printf("Error: malloc(%d) failed. Out of memory?\n", n);
   exit(EXIT_FAILURE);
 }
 return p;
}
```

```
int *p = safeMalloc(sizeof(int));
*p = 42;
```

Memory Deallocation: ‘Clean Up’

Dynamically allocated memory must be returned (i.e. deallocated) when it is no longer needed.

```
free(p);
/* Note: p has not changed! (call-by-value, remember?)
 * But, you cannot use *p any longer!
 */
```

Pointer Arithmetic

You can perform (simple) arithmetic with pointers:

- *sum* of a pointer and an integer
- *difference* between a pointer and and integer
- pointer *comparison*

Sum of a pointer and an integer

```
int *p = safeMalloc(100*sizeof(int));
```

The sum `p + n` (where `n` is an integer) has the same type as `p`.

So, an int-pointer

We can produce a pointer to the `n`th element of a memory block by simply adding `n` to the pointer `p`, i.e. `p + n`.

This pointer points `n` elements (not bytes!) further than `p`.

This is one of the reasons why pointers are typed (`int *`, `char *`, etc).

So, do not write `p + n*sizeof(int)`

This operation (+) is not defined for void-pointers!

Sum of a pointer and an integer

So, we can address the `n`th element of a memory block using

```
*(p + n)
```

This is the same as (preferred notation):

```
p[n]
```

Hence, `p[0]` and `*(p+0)` and `*p` are equivalent.

Hence, `p[i]` and `*(p+i)` and `i[p]` are all equivalent.

Hence, `&p[i]` and `p+i` are equivalent.

Arrays and Pointers

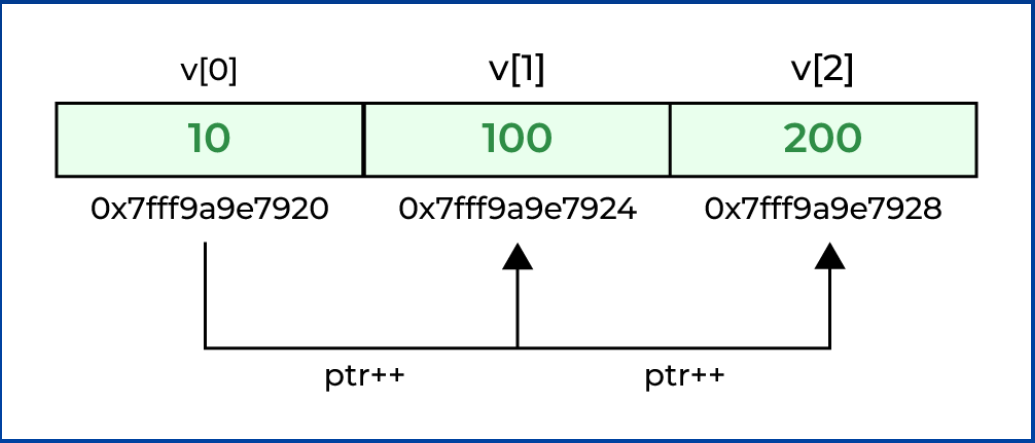
An array is actually a *typed constant* pointer that is initialised as a pointer that points to a memory block of exactly the right size.

```
int v[3];
int *ptr = &v[0];
```

`v` is a (constant!) `int`-pointer that points to a memory block that is reserved for 3 integers.

`ptr` is a (non-constant!) pointer to an `int`.

```
ptr = v;  /* ok */
v = ptr;  /* wrong */
```



Pointer Comparison

Two pointers of the same type, `p` and `q`, can be compared.

The comparison operators are: `<` , `>` , `<=` , `==` , `>=` , `!=`

BEWARE: if you compare two pointers, then you compare memory addresses (NOT the content of the addresses)!

```
int main () {
  int *p, *q, *r;
  p = safeMalloc(3*sizeof(int));
  q = safeMalloc(3*sizeof(int));

  *p = 0;
  *(p+1) = 1;
  p[2] = 2;

  printf ("p: %d %d %d\n", p[0], p[1], p[2]);

  r = p+3;
  while (p!=r) {
    *q++ = *p++;
  }
  q -= 3;
  p -= 3;

  printf ("p: %d %d %d\n", p[0], p[1], p[2]);
  printf ("q: %d %d %d\n", q[0], q[1], q[2]);
  return 0;
}
```

```
p: 0 1 2
p: 0 1 2
q: 0 1 2
```

Dangling References

Stack-data are deallocated automatically as soon as a function terminates.

So, NEVER return a pointer to a local variable!

```
int *f() {
   int i = 2;
   int *ptr = &i;
   printf("ptr points to %d\n", *ptr);  /* ok */
   return ptr;  /* DO NOT DO THIS! */
}

int main(int argc, char *argv[]) {
  int *ptr = f();
  printf("ptr points to %d\n", *ptr); /* wrong!!! */
  return 0;
}
```

Operations on Memory Blocks

Operations on blocks of memory use *generic (void)* pointers and are declared in the standard header file `string.h`.

A memory block can be resized to `len` bytes using:

```
void *realloc(void *ptr, int len);
```

A memory block of `len` bytes can be copied from `src` to `dest` (blocks should not overlap) using:

```
void *memcpy(void *dest, const void *src, int len);
```

A memory block of `len` bytes can be completely filled with a certain byte value using:

```
void *memset(void *s, int c, size_t n);
```

For example: `memset(s, 0, 100*sizeof(int));`

End of intermezzo

Grades: plan

- Assignment: write a program fragment that reads the grades of a number of students from the input and then prints a list with the grade of every student and the deviation relative to the mean grade.

- We number the students, starting with 0.
- We ask the user to first type the number of students.
- We create an array that has the right size.
- Read the grades from the input.
- Compute the mean.
- Print the list with grades and deviations.

Grades

```
int numStudents;
printf("Number of students: ");
scanf("%d", &numStudents);

float *grade = safeMalloc(numStudents*sizeof(float));

for (int s=0; s < numStudents; s++) {
  printf("Grade of student %d: ", s);
  scanf("%f", &grade[s]);
}

float sum = 0.0;
for (int s=0; s < numStudents; s++) {
  sum += grade[s];
}

float mean = sum/numStudents;

print("student\tgrade\tdeviation\n");
for (int s=0; s < numStudents; s++) {
  printf("%d\t%.1f\t%.1f\n", s, grade[s], grade[s] - mean);
}
free(grade);
```

Grades: merge loops

```
int numStudents;
printf("Number of students: ");
scanf("%d", &numStudents);

float *grade = safeMalloc(numStudents*sizeof(float));

float sum = 0.0;
for (int s=0; s < numStudents; s++) {
  printf("Grade of student %d: ", s);
  scanf("%f", &grade[s]);
  sum += grade[s];
}

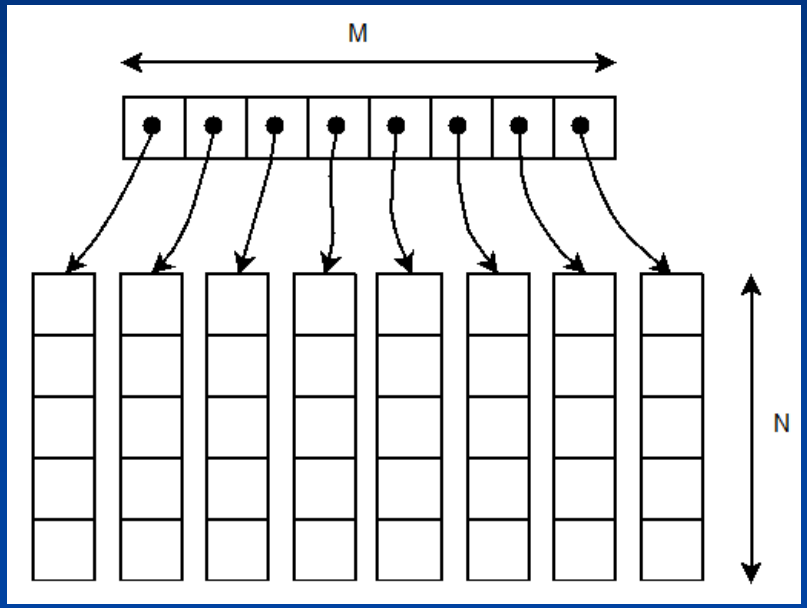
float mean = sum/numStudents;

print("student\tgrade\tdeviation\n");
for (int s=0; s < numStudents; s++) {
  printf("%d\t%.1f\t%.1f\n", s, grade[i], grade[i] - mean);
}

free(grade);
```

Making a dynamic 2D-array

```
/* make a 2D dynamic array */
int **makeIntArray2D(int width, int height) {
  int **arr = safeMalloc(height*sizeof(int *));
  for (int row=0; row < height; row++) {
    arr[row] = safeMalloc(width*sizeof(int));
  }
  return arr;
}
```



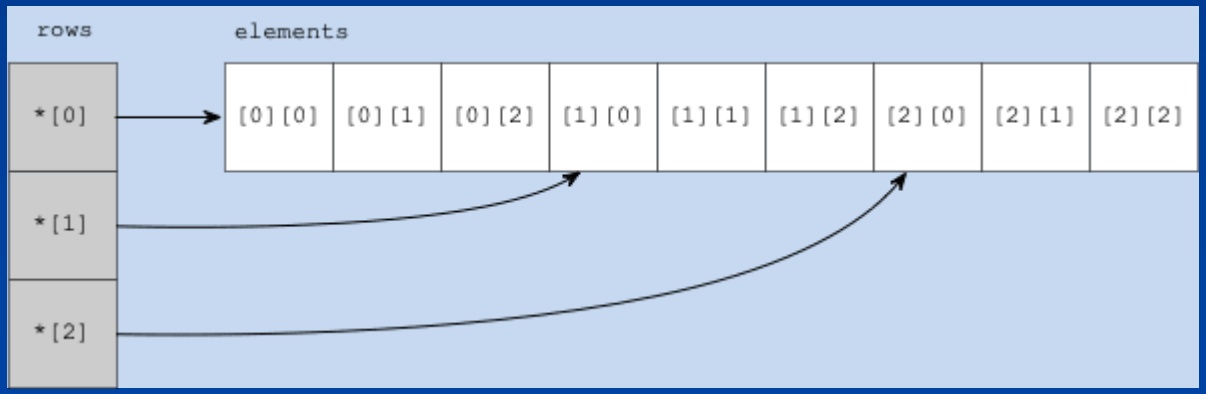

```
/* make dynamic N x M array */
int **arr = makeIntArray2D(N, M);
```

Freeing a dynamic 2D-array

```
/* free a 2D dynamic array */
void destroyIntArray2D(int **arr) {
  for (int row=0; row < height; row++) {
    free(arr[row]);
  }
  free(arr);
}
```

Making a dynamic 2D-array (alternative version)

```
/* make a 2D dynamic array */
int **makeIntArray2D(int width, int height) {
  int **arr = safeMalloc(height*sizeof(int *));
  arr[0] = safeMalloc(width*height*sizeof(int));
  for (int row=1; row < height; row++) {
    arr[row] = arr[row-1] + width;
  }
  return arr;
}
```



Freeing a dynamic 2D-array (alternative version)

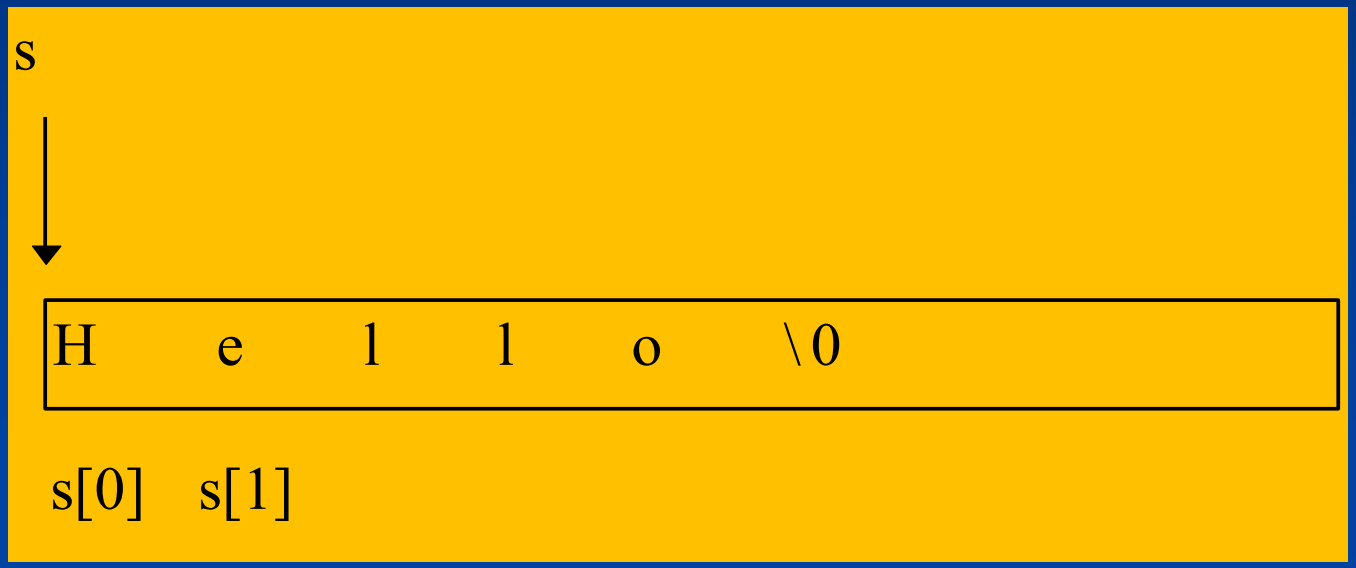
```
/* free a 2D dynamic array */
void destroyIntArray2D(int **arr) {
  free(arr[0]);
  free(arr);
}
```

Strings in C

In C, a string is used to store text.

- It is a consecutive sequence of characters between double quotes (e.g. `"underwear"`)
- A string is actually a character array: `char *s;`

A string is terminated by `'\0'` (null-character):



Strings in C

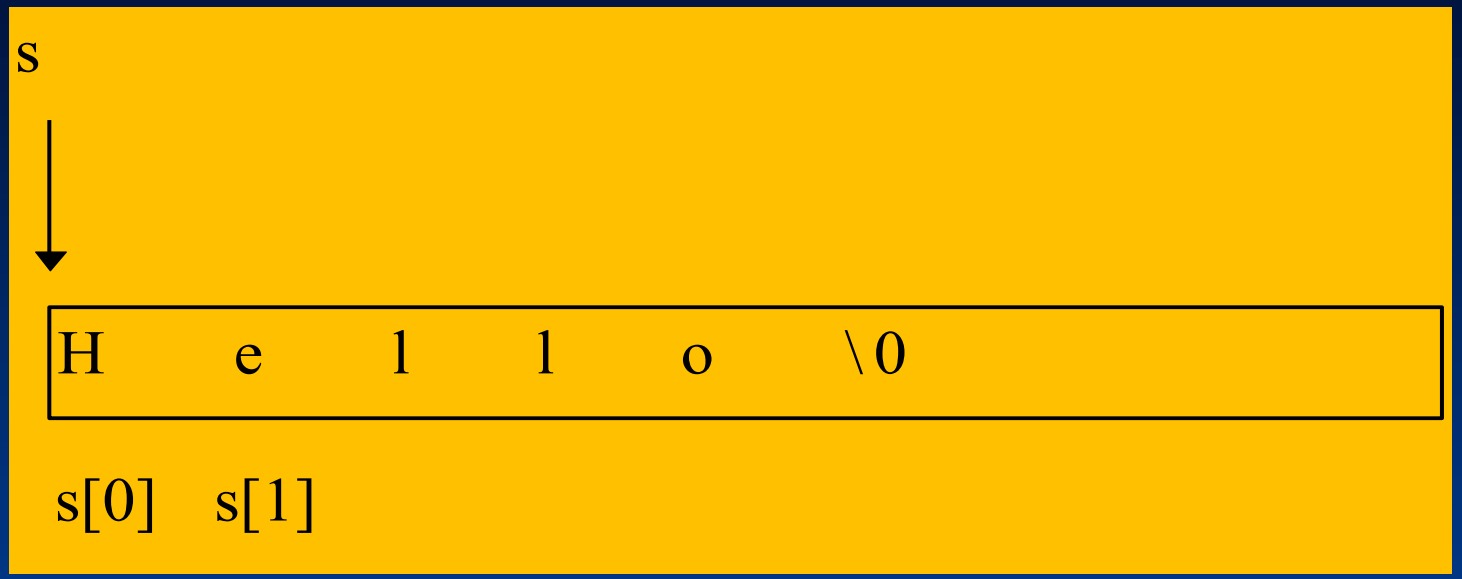
Strings are defined as pointers to characters:

```
char *s;
```

Beware! When you allocate memory for a string, you have to take into account that there must be space for the terminating null-character:

```
s = malloc((1+5)*sizeof(char));  // just enough to fit "hello"
```

Definition of a String



The string `s` has length 5; `"Hello"`

The string `s+1` (a suffix of `s`) has length 4; `"ello"`

The string `s+2` (a suffix of `s`) has length 3; `"llo"`

The string `s+5` (a suffix of `s`) has length 0; `""`

The string `s+6` is undefined!

String Parameters

C strings can be used as (formal) parameters, just like any other pointer/array:

```
void upperCaseLetter(char *s) {
    s[0] = toupper(s[0]);
}
```

Formatted String I/0

There are two formatted string I/O operations:

```
int scanf("%s", strargument);
int printf("%s", strargument);
```

The format specifier `%s` is used for string I/O.

`scanf` skips *white space!*

Strings/Tokens are separated by whitespace.

So, `scanf()` can only match a word (not a sentence)!

```
char name[32];
printf("Please, type your name: ");
scanf("%s", name);   /* NOTE: no ampersand (&) */
```

C String Operations

```
#include <string.h>
```

Length of a string:

```
int strlen(char *string);
```

`strlen("Hello")` **returns 5 (not 6, so without null character)**

Copy a string:

```
char *strcpy(char *dest, char *src);
```

Copy at most `n` characters:

```
char *strncpy(char *dest, char *src, int n);
```

If the length of `str` is less than `n`, then it is padded with a trailing null-character (i.e. `'\0'`).

Warning: `dest` need not be terminated by a null character!

C String Operations: `str(n)cat`

Concatenation ('append src to dest'):

```
char *strcat(char *dest, char *src);
```

Concatenation of at most `n` characters from `src`:

```
char *strncat(char *dest, char *src, int n);
```

C String Operations: comparison

Lexicographic comparison of **s1** and **s2**:

```
int strcmp(char *s1, char *s2);
```

Returns a negative value if **s1** is less than **s2**, 0 if the strings are equal, or a positive value if **s1** is greater than **s2**.

Lexicographic comparison of the first **n** characters of **s1** and **s2**:

```
int strncmp(char *s1, char *s2, int n);
```

Character Functions

Character classification:

`int isalnum(int c)`	is `c` alphanumeric?
`int isalpha(int c)`	is `c` a letter from the alphabet?
`int islower(int c)`	is `c` a lower-case letter?
`int isupper(int c)`	is `c` an upper-case letter?
`int isdigit(int c)`	is `c` a digit?
`int isprint(int c)`	is `c` printable? (no control character)

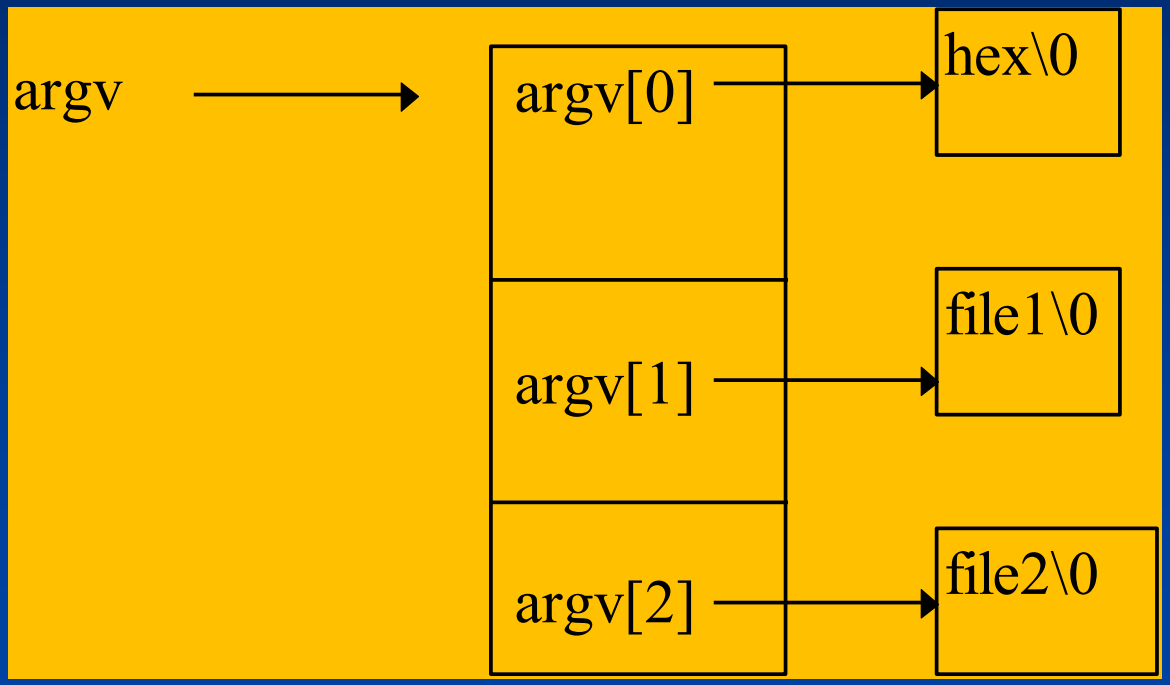
Conversion:

```
int tolower(int c)
int toupper(int c)
```

Main Function: Arguments

```
int main(int argc, char *argv[]);
int main(int argc, char **argv);
```

Commandline:
hex file file2



Argv: passing data to main

Each command line token is stored as a separate *element* in the *array* **`argv`**.

The name of the program is stored in **`argv[0]`**.

The first token passed to the program is stored in **`argv[1]`**, the second token in **`argv[2]`**, etc.

`argc` stores the number of tokens (including the program name) passed in.

argc/argv example

```
int main (int argc, char* argv[]){
  printf("Commandline:");
  for (int i=0; i < argc; i++) {
    printf(" %s", argv[i]);
  }
  printf("\n");
  return 0;
}
```

argc/argv cont.

Note that argv is an array of strings.

So, as an example, we cannot simply pass two numbers (e.g. a.out 2 5) and add them.

First, we need to convert the arguments to `int`s.

```
int main(int argc, char *argv[]) {
  int a, b;
  a = atoi(argv[1]);  /* convert to integer */
  b = atoi(argv[2]);
  printf("%d\n", a + b);
  return 0;
}
```



End week 4